

Software Requirements Specifications

NAVISENCE

Environment Detection for Blind People

Project Code: EDBP-2025

Supervisor

Prof. Muhammad Fahad

Project Manager

Dr. Muhammad Ilyas

Project Team

Muhammad Ali Raza Ansari (BSCS51F22S008) — Team Leader

Muhammad Abdullah Zammad (BSCS51F22S036) — Team Member

Muhammad Hassan Javed (BSCS51F22S040) — Team Member

Submission Date

October 20, 2025



Project Supervisor's Signature

Document Information

Customer	Computer Science Department
Project	Navisence — Environment Detection for Blind People
Document Type	Software Requirements Specifications (SRS)
Document Version	1.3
Identifier	BSCS-51-F-22-S
Status	Final
Author(s)	Muhammad Ali Raza Ansari (BSCS51F22S008) Muhammad Abdullah Zammad (BSCS51F22S036) Muhammad Hassan Javed (BSCS51F22S040)
Supervisor	Prof. Muhammad Fahad
Approver(s)	Dr. Muhammad Ilyas (Project Manager)
Issue Date	October 15, 2025
Distribution	1. Internal Advisor 2. Project Manager 3. Project Office

Definition of Terms, Acronyms, and Abbreviations

Term / Acronym	Description
TFLite	TensorFlow Lite — lightweight on-device machine learning inference framework
RTOD	Real-Time Object Detection
TTS	Text-To-Speech — offline audio feedback engine

YOLO	You Only Look Once — real-time object detection algorithm
YOLOv8	Version 8 of the YOLO model by Ultralytics, used for object detection
Flutter	A cross-platform mobile application development framework by Google
EDBP	Environment Detection for Blind People
SRS	Software Requirements Specifications
FR	Functional Requirement
NFR	Non-Functional Requirement
SC	System Constraint
AS	Assumption

Table of Contents

1. Introduction	5
1.1 Purpose of Document	5
1.2 Project Overview	5
1.3 Scope	5
2. Overall System Description	5
2.1 User Characteristics	6
2.2 Operating Environment	6
2.3 System Constraints	6
3. External Interface Requirements	7
3.1 Hardware Interfaces	7
3.2 Software Interfaces	7
3.3 Communications Interfaces	7
4. Functional Requirements	7
5. Non-Functional Requirements	8
6. Assumptions and Dependencies	8
6.1 Assumptions	8
6.2 Dependencies	9
7. References	9

1. Introduction

1.1 Purpose of Document

This document defines the objectives, scope, functional requirements, and non-functional requirements of the Navisence system, also known as the Environment Detection for Blind People (EDBP). The system leverages AI-based (Smart-phone application) object detection and fully offline audio feedback to assist visually impaired individuals in identifying and navigating around objects in both indoor and outdoor environments. This document is intended for internal advisors, project managers, developers, evaluators, and all stakeholders involved in the project's design, development, and assessment. The structure and content of this Software Requirements Specification (SRS) follow the standard guidelines by IEEE [\[1\]](#).

1.2 Project Overview

Navisence is an AI-powered, fully offline mobile application designed to assist visually impaired users in perceiving their surroundings. The system uses the smartphone's built-in camera to detect objects in real time using the YOLOv8 model [\[2\]](#). The mobile application is built with the Flutter framework [\[3\]](#), and spoken audio feedback is delivered to the user via an on-device Text-to-Speech (TTS) engine using the flutter_tts package[\[4\]](#), indicating the detected object, its direction (left, center, or right), and its approximate distance (near, medium, or far). All processing occurs entirely on-device without requiring any internet connection, making the system reliable, accessible, and suitable for use in any environment.

1.3 Scope

Included:

- Real-time object detection using a smartphone
- Fully offline operation — no internet connection required
- Audio feedback using offline Text-to-Speech (TTS)
- Direction identification (Left, Center, Right)
- Distance estimation (Near, Medium, Far)
- Continuous real-time system operation
- Simple and accessible mobile user interface

Not Included:

- Exact metric distance measurement
- Navigation or path planning features
- IoT devices or external hardware integration
- Facial recognition functionality
- Text reading or Optical Character Recognition (OCR)

2. Overall System Description

Navisence is an AI-based offline mobile system designed to assist blind individuals in recognizing objects in their surroundings through real-time audio feedback. The system uses the smartphone camera to

continuously capture frames, processes them entirely on-device using a YOLOv8 TensorFlow Lite model, and delivers spoken output to guide the user safely and independently across both indoor and outdoor environments.

2.1 User Characteristics

- **Primary Users:** Blind individuals who rely entirely on audio feedback for object recognition and spatial navigation. These users require a minimal-interaction, hands-free interface with timely and accurate audio output.
- **Secondary Users:** Developers, testers, and system administrators responsible for application installation, model updates, and ongoing system maintenance.
- **Critical Users:** Blind individuals are the most critical user group, as system reliability and detection accuracy directly impact their safety and independence.

2.2 Operating Environment

The system is developed and deployed as a mobile application on the Android platform. The operating environment consists of the following technologies:

- **Frontend:** Flutter — cross-platform mobile application framework
- **AI Framework:** TensorFlow Lite (TFLite) — on-device inference
- **Object Detection Model:** YOLOv8 — trained on a custom dataset
- **Audio Output:** Offline Text-to-Speech (TTS) Engine
- **Platform:** Android (Mobile)

2.3 System Constraints

- **SC-1 — Software:** Real-time detection performance may be affected by the device's processing speed and available memory resources.
- **SC-2 — Hardware:** The system requires a functioning smartphone camera, audio output capability (speaker or earphones), and sufficient RAM to execute all features.
- **SC-3 — Linguistics:** The system currently supports English audio output only; additional languages may be incorporated in future releases.
- **SC-4 — Legal:** The system must comply with applicable data privacy regulations. No user images or personal data shall be stored or transmitted.
- **SC-5 — Environmental:** Detection accuracy may degrade in low-light conditions or with excessive camera movement.
- **SC-6 — User:** All user interaction relies solely on audio feedback. Minimal visual elements are included only for initial setup and developer debugging.
- **SC-7 — Model Size:** The TFLite model must remain under 30 MB to operate efficiently on standard Android devices.
- **SC-8 — Object Scope:** The initial release supports a defined set of common objects. The model can be expanded through future updates to include additional object classes.

3. External Interface Requirements

The Navisence system interfaces with the smartphone camera and audio output hardware, utilizes TensorFlow Lite and Flutter software frameworks, and delivers all output exclusively through the offline TTS engine.

3.1 Hardware Interfaces

- Camera: Captures real-time images of the surrounding environment for on-device object detection processing.
- Earphones / Speaker: Delivers all audio feedback and spoken alerts directly to the user.
- Android Smartphone: The primary processing unit on which all detection, inference, and audio output tasks execute entirely on-device.

3.2 Software Interfaces

- Flutter: Provides the cross-platform mobile application frontend, delivering a minimal and accessible interface suited for blind users.
- TensorFlow Lite (TFLite): Executes the YOLOv8 object detection model on-device for real-time, fully offline inference.
- YOLOv8 (Custom-trained): Performs object detection and classification on preprocessed camera frames.
- Android OS: The target operating system for execution of the application.

3.3 Communications Interfaces

All system processing occurs entirely on-device. Camera frames are passed directly to the TFLite inference engine for object detection. No external server, cloud service, or internet connection is required for any core functionality. Audio feedback is delivered locally through the device speaker or connected earphones via the on-device TTS engine.

4. Functional Requirements

- FR-1 — Object Detection: The system shall capture images using the smartphone camera and identify surrounding objects using the YOLOv8 model executed via TensorFlow Lite, entirely on-device without internet connectivity.
- FR-2 — Image Preprocessing: The system shall prepare each captured frame by resizing and enhancing image clarity before model inference.
- FR-3 — Direction Identification: The system shall determine and report whether detected objects are positioned to the left, center, or right of the user based on the bounding box position within the camera frame.
- FR-4 — Distance Estimation: The system shall estimate the proximity of detected objects and classify them as Near, Medium, or Far based on the relative bounding box area within the camera frame.

- FR-5 — Audio Feedback: The system shall convert detected objects, their direction, and estimated distance into spoken words using the offline on-device Text-to-Speech (TTS) engine.
- FR-6 — Real-Time Processing: The system shall continuously process camera frames and deliver feedback in real time to support safe and uninterrupted user navigation.
- FR-7 — Confidence Filtering: The system shall only announce detected objects when the model confidence score exceeds a predefined threshold to minimize false positives.
- FR-8 — Offline Operation: The system shall perform all core detection and feedback functions entirely without internet connectivity, using only on-device resources.
- FR-9 — User Interface: The mobile application, built with Flutter, shall provide a simple, minimal-interaction interface fully accessible to blind users.
- FR-10 — Language Support: The system shall deliver audio feedback in English by default, with the capability to support additional languages through future model and TTS updates.

5. Non-Functional Requirements

- NFR-1 — Performance: The system shall deliver object detection results and audio feedback with minimal latency, targeting under one second per inference cycle, depending on device hardware capability.
- NFR-2 — Accuracy: The system shall maintain acceptable detection accuracy under standard indoor and outdoor environmental and lighting conditions.
- NFR-3 — Reliability: The system shall operate continuously and stably without crashing or freezing during extended usage sessions.
- NFR-4 — Safety: The system shall prioritize user safety by ensuring all audio feedback is timely, accurate, and free from misleading or erroneous information.
- NFR-5 — Privacy: The system shall not store, transmit, or share any captured images, inference results, or personal user data at any point during operation.
- NFR-6 — Resource Efficiency: The system shall operate efficiently with respect to battery consumption, RAM usage, and CPU load to ensure prolonged usability on standard Android devices.

6. Assumptions and Dependencies

6.1 Assumptions

- AS-1: Users will have access to a compatible Android device equipped with a functional camera and audio output capability (speaker or earphones).

- AS-2: Third-party frameworks, including TensorFlow Lite and Flutter, will remain stable, available, and compatible with future software updates throughout the project lifecycle.
- AS-3: Hardware components, specifically the camera and audio output device, will function correctly and meet the minimum performance standards required for real-time detection.
- AS-4: The offline TTS engine will be available and functional on the target Android device without requiring internet access.
- AS-5: The YOLO nano v8 TFLite model will be packaged and embedded within the application for fully offline use.

6.2 Dependencies

- The project depends on the stability of TensorFlow Lite and Flutter (version 3.0 or higher).
- The offline TTS engine and YOLOv8 TFLite model are important components; licensing changes or version incompatibilities may affect system functionality.
- System accuracy is dependent on the quality and completeness of the custom-trained YOLOv8 dataset and annotation.
- Model training depends on sufficient dataset collection, preprocessing, and annotation for all target object classes.
- TFLite integration with Flutter and the mobile camera module is a critical dependency affecting the development timeline.
- Runtime performance depends on the hardware capabilities (CPU and RAM) of the target Android device.
- The application is developed, installed, and tested directly on Android mobile devices.

7. References

- [1] Wikipedia Contributors, "Software Requirements Specification," *Wikipedia, The Free Encyclopedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Software_requirements_specification
- [2] Ultralytics, "YOLOv8 Documentation," 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolov8>
- [3] Flutter Development Team, "Flutter Documentation," 2025. [Online]. Available: <https://docs.flutter.dev/>
- [4] D. Shababb, "flutter_tts," *pub.dev*, 2025. [Online]. Available: https://pub.dev/packages/flutter_tts